



Lab Assignment - 03

Course Title: Database System

Course Code: CSE370

Section: 05

Semester: 5

Name: Sk Md Saad Bin Sohan

ID: 221

Department: Computer Science and Engineering

Submitted To:

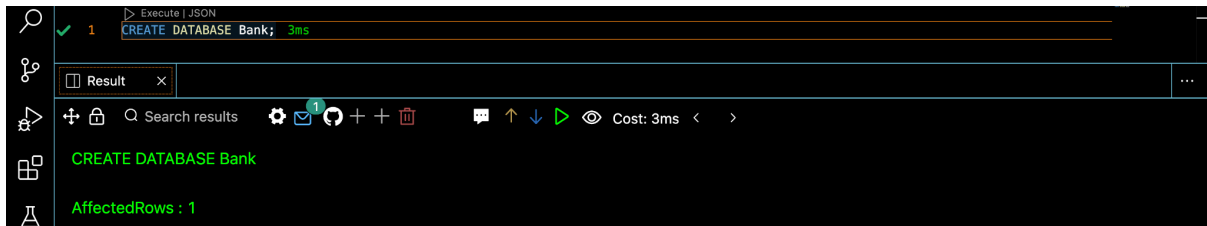
Mohammad Sayeem Sadat [MSDH], Farden Ehsan Khan [FEK]

Lecturer, Department of CSE,
BRAC University

Submitted on:

17 February 2024

Screenshots of the queries of the given database as they were given:



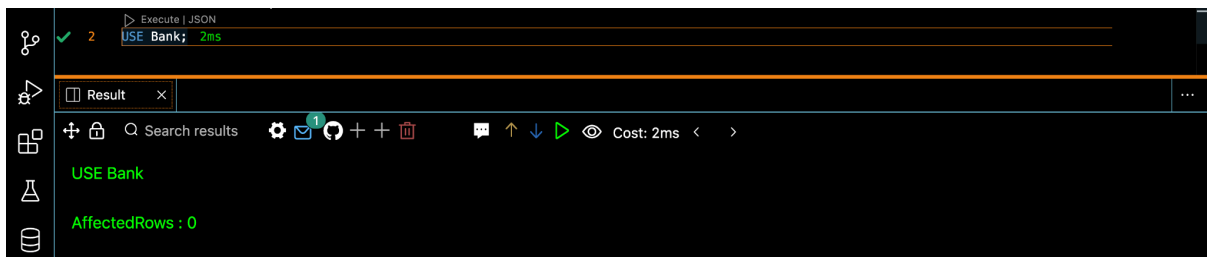
1 > Execute | JSON
`CREATE DATABASE Bank;` 3ms

Result x

Search results 1 Cost: 3ms

`CREATE DATABASE Bank`

AffectedRows : 1



2 > Execute | JSON
`USE Bank;` 2ms

Result x

Search results 1 Cost: 2ms

`USE Bank`

AffectedRows : 0



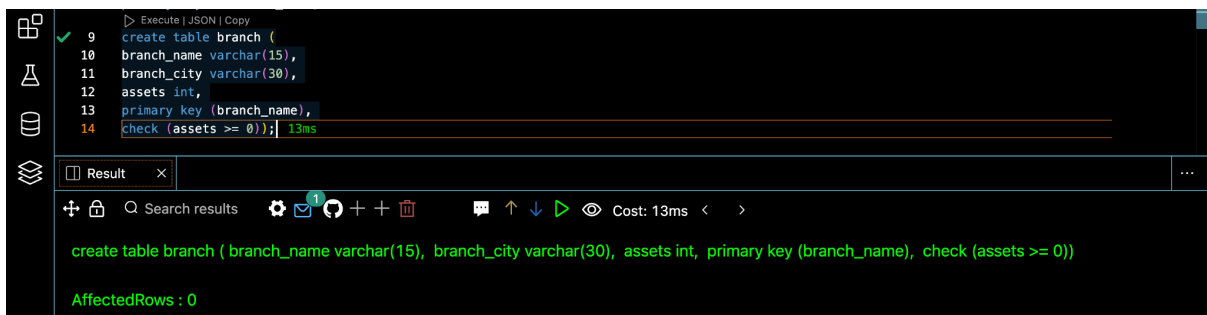
3 > Execute | JSON | Copy
4 `create table customer (`
5 `customer_id varchar(10) not null,`
6 `customer_name varchar(20) not null,`
7 `customer_street varchar(30),`
8 `customer_city varchar(30),`
9 `primary key (customer_id));` 18ms

Result x

Search results 1 Cost: 18ms

`create table customer ( customer_id varchar(10) not null, customer_name varchar(20) not null, customer_street varchar(30), customer_city varchar(30), primary key (customer_id))`

AffectedRows : 0



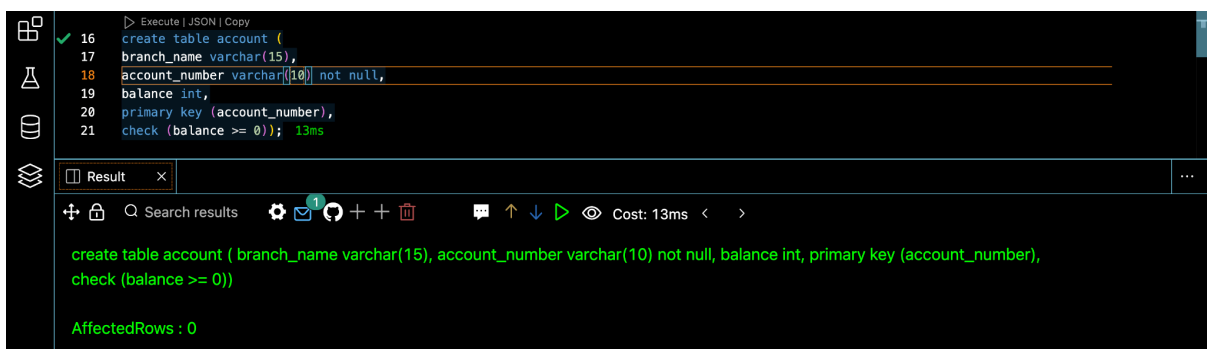
9 > Execute | JSON | Copy
10 `create table branch (`
11 `branch_name varchar(15),`
12 `branch_city varchar(30),`
13 `assets int,`
14 `primary key (branch_name),`
15 `check (assets >= 0));` 13ms

Result x

Search results 1 Cost: 13ms

`create table branch ( branch_name varchar(15), branch_city varchar(30), assets int, primary key (branch_name), check (assets >= 0))`

AffectedRows : 0



16 > Execute | JSON | Copy
17 `create table account (`
18 `branch_name varchar(15),`
19 `account_number varchar(10) not null,`
20 `balance int,`
21 `primary key (account_number),`
22 `check (balance >= 0));` 13ms

Result x

Search results 1 Cost: 13ms

`create table account ( branch_name varchar(15), account_number varchar(10) not null, balance int, primary key (account_number), check (balance >= 0))`

AffectedRows : 0

```
22
23 > Execute | JSON | Copy
24 create table loan (
25   loan_number varchar(10) not null,
26   branch_name varchar(15),
27   amount int,
28   primary key (loan_number)); 10ms
```

Result

create table loan (loan_number varchar(10) not null, branch_name varchar(15), amount int, primary key (loan_number))

AffectedRows : 0

```
29 > Execute | JSON | Copy
30 create table depositor (
31   customer_id varchar(10) not null,
32   account_number varchar(10) not null,
33   primary key (customer_id,account_number),
34   foreign key (customer_id) references customer(customer_id),
35   foreign key (account_number) references account(account_number)); 14ms
```

Result

create table depositor (customer_id varchar(10) not null, account_number varchar(10) not null, primary key (customer_id,account_number), foreign key (customer_id) references customer(customer_id), foreign key (account_number) references account(account_number))

AffectedRows : 0

```
36 > Execute | JSON | Copy
37 create table borrower (
38   customer_id varchar(10) not null,
39   loan_number varchar(10) not null,
40   primary key (customer_id, loan_number),
41   foreign key (customer_id) references customer(customer_id),
42   foreign key (loan_number) references loan(loan_number)); 14ms
```

Result

create table borrower (customer_id varchar(10) not null, loan_number varchar(10) not null, primary key (customer_id, loan_number), foreign key (customer_id) references customer(customer_id), foreign key (loan_number) references loan(loan_number))

AffectedRows : 0

```
43 > Execute | JSON | Copy
44 insert into customer values
45   ('C-101','Jones', 'Main', 'Harrison'),
46   ('C-201','Smith', 'North', 'Rye'),
47   ('C-211','Hayes', 'Main', 'Harrison'),
48   ('C-212','Curry', 'North', 'Rye'),
49   ('C-215','Lindsay', 'Park', 'Pittsfield'),
50   ('C-220','Turner', 'Putnam', 'Stamford'),
51   ('C-222','Williams', 'Nassau', 'Princeton'),
52   ('C-225','Adams', 'Spring', 'Pittsfield'),
53   ('C-226','Johnson', 'Alma', 'Palo Alto'),
54   ('C-233','Glenn', 'Sand Hill', 'Woodside'),
55   ('C-234','Brooks', 'Senator', 'Brooklyn'),
56   ('C-255','Green', 'Walnut', 'Stamford');
```

Result

insert into customer values ('C-101','Jones', 'Main', 'Harrison'), ('C-201','Smith', 'North', 'Rye'), ('C-211','Hayes', 'Main', 'Harrison'), ('C-212','Curry', 'North', 'Rye'), ('C-215','Lindsay', 'Park', 'Pittsfield'), ('C-220','Turner', 'Putnam', 'Stamford'), ('C-222','Williams', 'Nassau', 'Princeton'), ('C-225','Adams', 'Spring', 'Pittsfield'), ('C-226','Johnson', 'Alma', 'Palo Alto'), ('C-233','Glenn', 'Sand Hill', 'Woodside'), ('C-234','Brooks', 'Senator', 'Brooklyn'), ('C-255','Green', 'Walnut', 'Stamford')

AffectedRows : 12

56

> Execute | JSON | Copy

✓ 57 insert into branch values

58 ('Downtown', 'Brooklyn',9000000),

59 ('Redwood', 'Palo Alto',2100000),

60 ('Perryridge', 'Horseneck',1700000),

61 ('Mianus', 'Horseneck',400000),

62 ('Round Hill', 'Horseneck',8000000),

63 ('Pownal', 'Bennington',300000),

64 ('North Town', 'Rye',3700000),

65 ('Brighton', 'Brooklyn',7100000); 8ms

66

Result

Search results

Cost: 8ms

insert into branch values ('Downtown','Brooklyn',9000000), ('Redwood','Palo Alto',2100000), ('Perryridge','Horseneck',1700000), ('Mianus','Horseneck',400000), ('Round Hill','Horseneck',8000000), ('Pownal','Bennington',300000), ('North Town','Rye',3700000), ('Brighton','Brooklyn',7100000)

AffectedRows : 8

67

> Execute | JSON | Copy

✓ 67 insert into account values

68 ('Downtown','A-101',500),

69 ('Mianus','A-215',700),

70 ('Perryridge','A-102',400),

71 ('Round Hill','A-305',350),

72 ('Brighton','A-201',900),

73 ('Redwood','A-222',700),

74 ('Brighton','A-217',750); 6ms

75

Result

Search results

Cost: 6ms

insert into account values ('Downtown','A-101',500), ('Mianus','A-215',700), ('Perryridge','A-102',400), ('Round Hill','A-305',350), ('Brighton','A-201',900), ('Redwood','A-222',700), ('Brighton','A-217',750)

AffectedRows : 7

76

> Execute | JSON | Copy

✓ 77 insert into loan values

78 ('L-17', 'Downtown', 1000),

79 ('L-23', 'Redwood', 2000),

80 ('L-15', 'Perryridge', 1500),

81 ('L-14', 'Downtown', 1500),

82 ('L-93', 'Mianus', 500),

83 ('L-11', 'Round Hill', 900),

84 ('L-16', 'Perryridge', 1300); 8ms

85

Result

Search results

Cost: 8ms

insert into loan values ('L-17','Downtown', 1000), ('L-23','Redwood', 2000), ('L-15','Perryridge', 1500), ('L-14','Downtown', 1500), ('L-93','Mianus', 500), ('L-11','Round Hill', 900), ('L-16','Perryridge', 1300)

AffectedRows : 7

86

Execute | JSON | Copy

87 insert into depositor values

88 ('C-226', 'A-101'),

89 ('C-201', 'A-215'),

90 ('C-211', 'A-102'),

91 ('C-220', 'A-305'),

92 ('C-226', 'A-201'),

93 ('C-101', 'A-217'),

94 ('C-215', 'A-222'); 6ms

95

Result

Search results

Cost: 6ms

insert into depositor values ('C-226', 'A-101'), ('C-201', 'A-215'), ('C-211', 'A-102'), ('C-220', 'A-305'), ('C-226', 'A-201'), ('C-101', 'A-217'), ('C-215', 'A-222')

AffectedRows : 7

96

Execute | JSON | Copy

97 insert into borrower values

98 ('C-101', 'L-17'),

99 ('C-201', 'L-23'),

100 ('C-211', 'L-15'),

101 ('C-226', 'L-14'),

102 ('C-212', 'L-93'),

103 ('C-201', 'L-11'),

104 ('C-222', 'L-17'),

105 ('C-225', 'L-16'); 11ms

Result

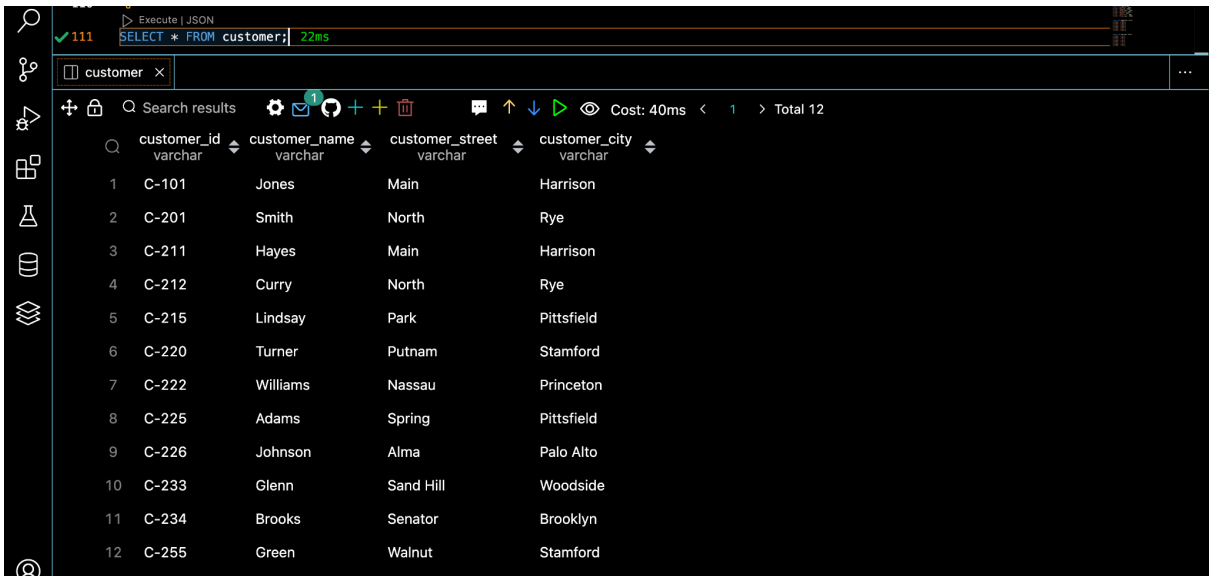
Search results

Cost: 11ms

insert into borrower values ('C-101', 'L-17'), ('C-201', 'L-23'), ('C-211', 'L-15'), ('C-226', 'L-14'), ('C-212', 'L-93'), ('C-201', 'L-11'), ('C-222', 'L-17'), ('C-225', 'L-16')

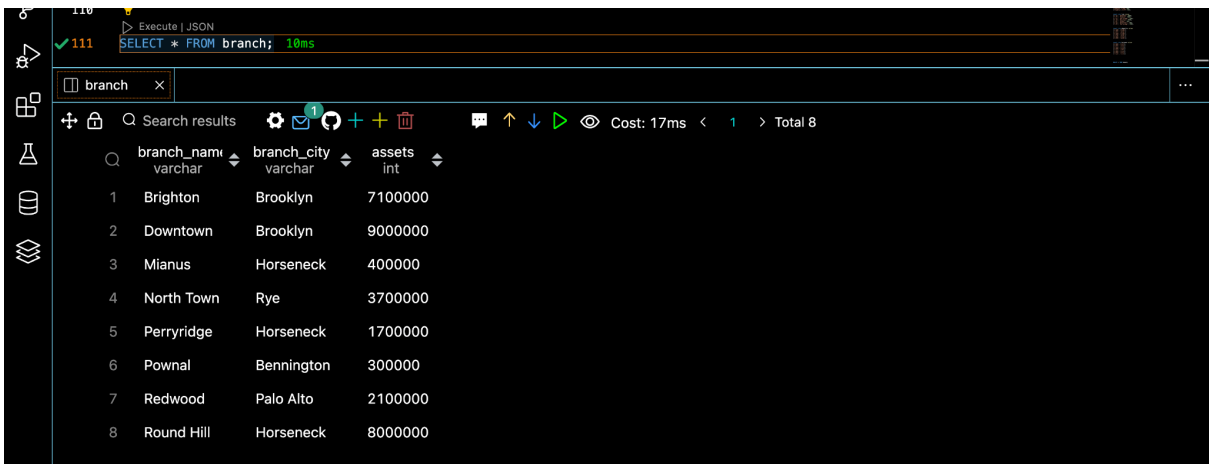
AffectedRows : 8

Table-1 customer:



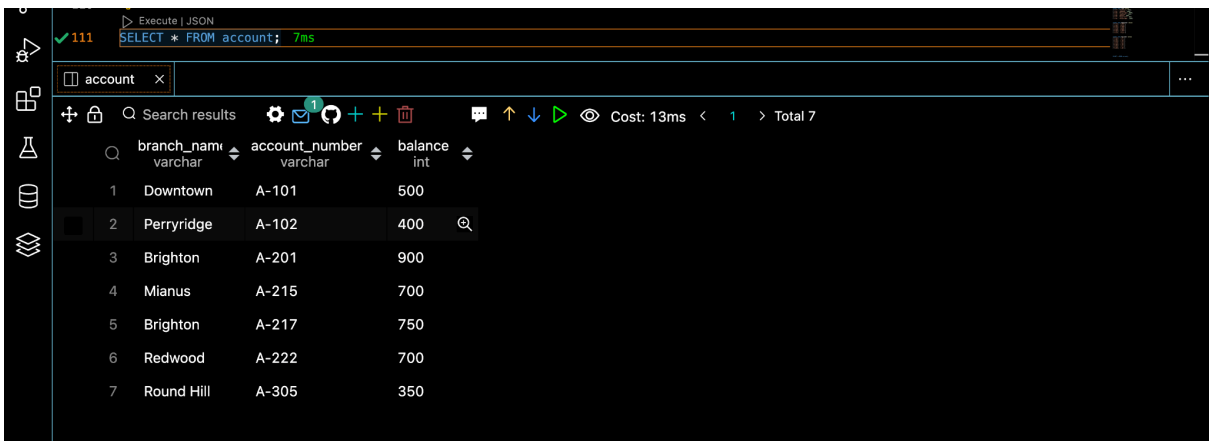
	customer_id varchar	customer_name varchar	customer_street varchar	customer_city varchar
1	C-101	Jones	Main	Harrison
2	C-201	Smith	North	Rye
3	C-211	Hayes	Main	Harrison
4	C-212	Curry	North	Rye
5	C-215	Lindsay	Park	Pittsfield
6	C-220	Turner	Putnam	Stamford
7	C-222	Williams	Nassau	Princeton
8	C-225	Adams	Spring	Pittsfield
9	C-226	Johnson	Alma	Palo Alto
10	C-233	Glenn	Sand Hill	Woodside
11	C-234	Brooks	Senator	Brooklyn
12	C-255	Green	Walnut	Stamford

Table-2 branch:



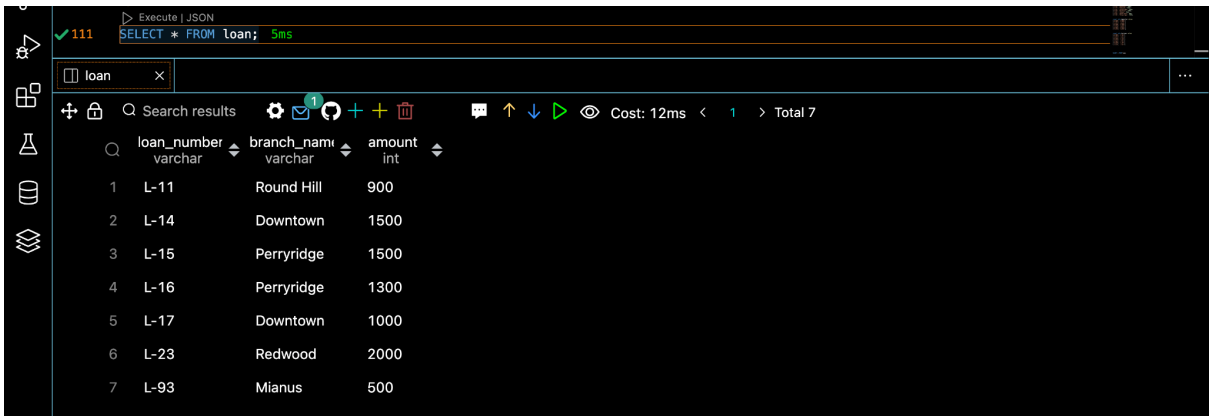
	branch_name varchar	branch_city varchar	assets int
1	Brighton	Brooklyn	7100000
2	Downtown	Brooklyn	9000000
3	Mianus	Horseneck	400000
4	North Town	Rye	3700000
5	Perryridge	Horseneck	1700000
6	Pownal	Bennington	300000
7	Redwood	Palo Alto	2100000
8	Round Hill	Horseneck	8000000

Table-3 account:



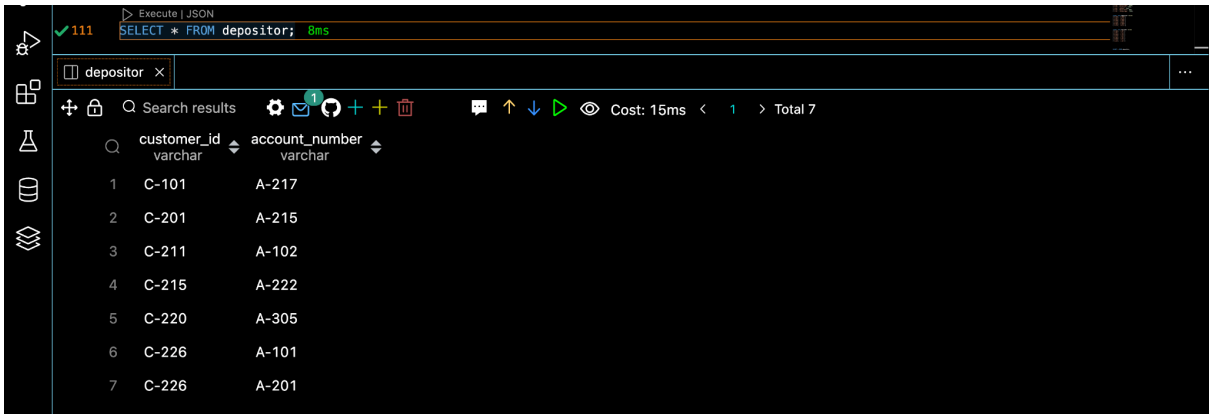
	branch_name varchar	account_number varchar	balance int
1	Downtown	A-101	500
2	Perryridge	A-102	400
3	Brighton	A-201	900
4	Mianus	A-215	700
5	Brighton	A-217	750
6	Redwood	A-222	700
7	Round Hill	A-305	350

Table-4 loan:



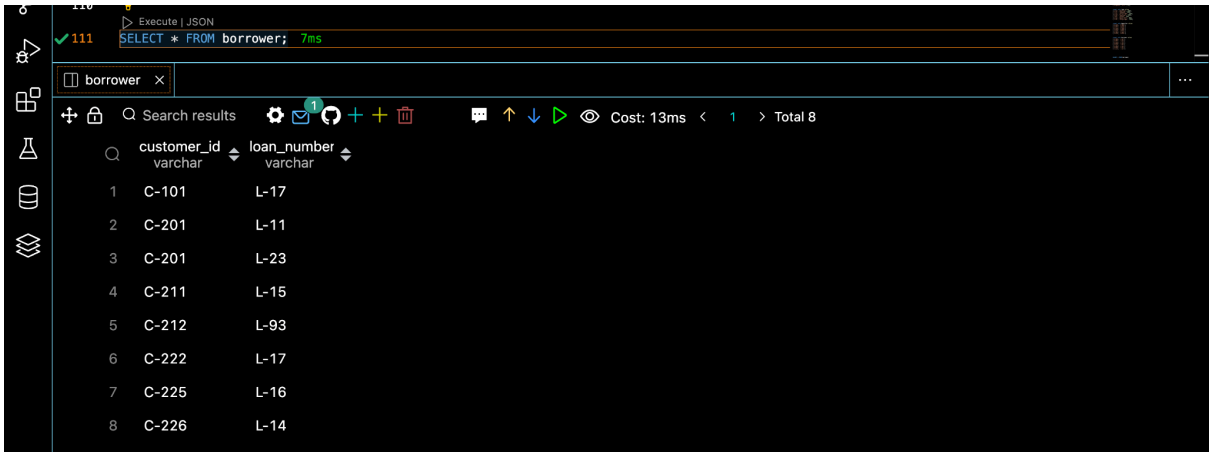
	loan_number varchar	branch_name varchar	amount int
1	L-11	Round Hill	900
2	L-14	Downtown	1500
3	L-15	Perryridge	1500
4	L-16	Perryridge	1300
5	L-17	Downtown	1000
6	L-23	Redwood	2000
7	L-93	Mianus	500

Table-5 depositor:



	customer_id varchar	account_number varchar
1	C-101	A-217
2	C-201	A-215
3	C-211	A-102
4	C-215	A-222
5	C-220	A-305
6	C-226	A-101
7	C-226	A-201

Table-6 borrower:

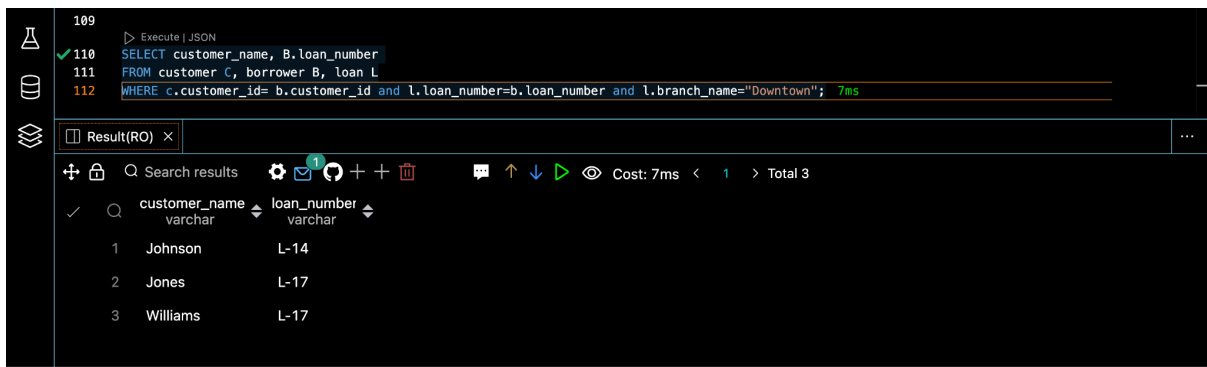


	customer_id varchar	loan_number varchar
1	C-101	L-17
2	C-201	L-11
3	C-201	L-23
4	C-211	L-15
5	C-212	L-93
6	C-222	L-17
7	C-225	L-16
8	C-226	L-14

Question-1: Find the name and loan number of all customers having a loan at the Downtown branch.

Query:

```
SELECT customer_name, B.loan_number
FROM customer C, borrower B, loan L
WHERE c.customer_id= b.customer_id and
l.loan_number=b.loan_number and l.branch_name="Downtown";
```



The screenshot shows a SQL query execution interface. The query is as follows:

```
110 SELECT customer_name, B.loan_number
111 FROM customer C, borrower B, loan L
112 WHERE c.customer_id= b.customer_id and l.loan_number=b.loan_number and l.branch_name="Downtown"; 7ms
```

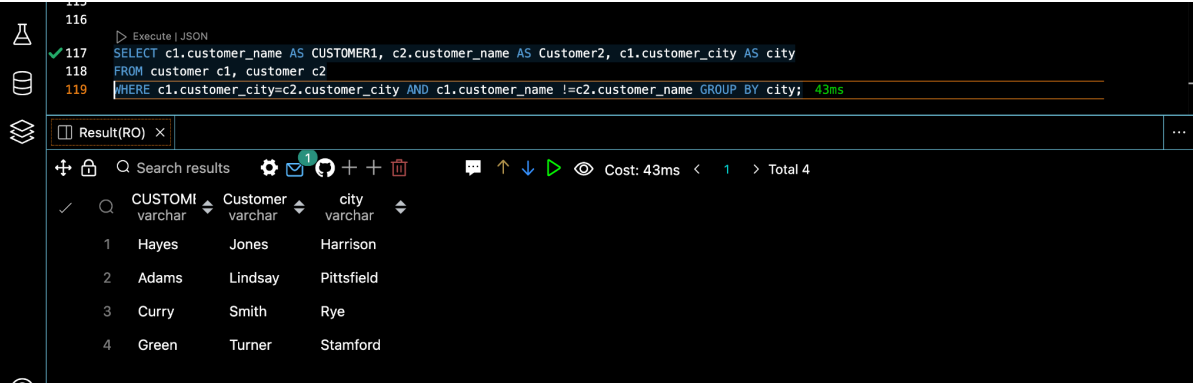
The results are displayed in a table with the following columns: customer_name (varchar) and loan_number (varchar). The results are as follows:

	customer_name	loan_number
1	Johnson	L-14
2	Jones	L-17
3	Williams	L-17

Question-2: Find all the possible pairs of customers who are from the same city.
show in the format Customer1, Customer2, City.

Query:

```
SELECT c1.customer_name AS CUSTOMER1, c2.customer_name AS  
Customer2, c1.customer_city AS city  
FROM customer c1, customer c2  
WHERE c1.customer_city=c2.customer_city AND c1.customer_name  
!=c2.customer_name GROUP BY city;
```



The screenshot shows a SQL query execution interface. The query is as follows:

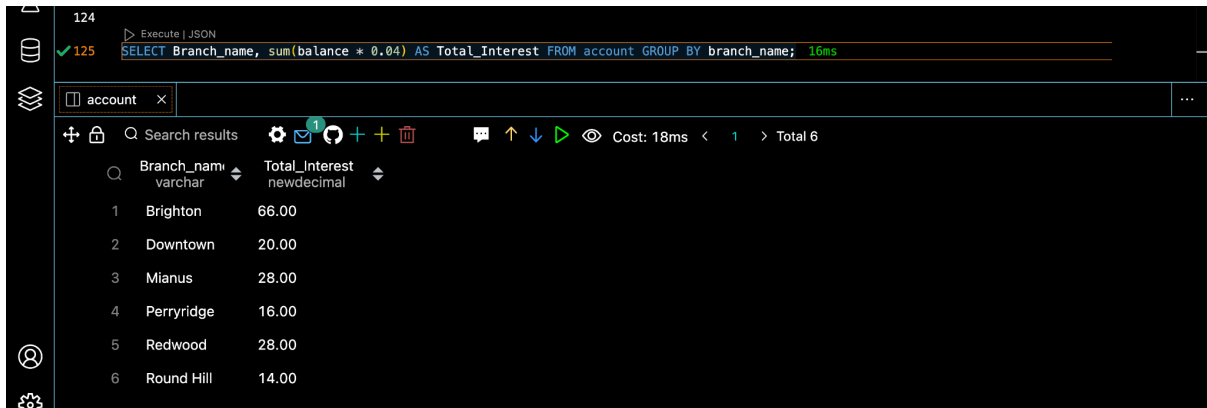
```
117 SELECT c1.customer_name AS CUSTOMER1, c2.customer_name AS Customer2, c1.customer_city AS city  
118 FROM customer c1, customer c2  
119 WHERE c1.customer_city=c2.customer_city AND c1.customer_name !=c2.customer_name GROUP BY city;
```

The results are displayed in a table with 4 rows and 3 columns: CUSTOMER1, Customer2, and city. The data is as follows:

	CUSTOMER1	Customer2	city
1	Hayes	Jones	Harrison
2	Adams	Lindsay	Pittsfield
3	Curry	Smith	Rye
4	Green	Turner	Stamford

Question-3: If the bank gives out 4% interest to all accounts, show the total interest across each branch. Print Branch_name, Total_Interest.

```
SELECT Branch_name, sum(balance * 0.04) AS Total_Interest FROM account GROUP BY branch_name;
```



The screenshot shows a database query execution interface. At the top, a SQL query is entered: `SELECT Branch_name, sum(balance * 0.04) AS Total_Interest FROM account GROUP BY branch_name;`. Below the query, the results are displayed in a table. The table has two columns: `Branch_name` (varchar) and `Total_Interest` (newdecimal). The results are as follows:

	Branch_name	Total_Interest
1	Brighton	66.00
2	Downtown	20.00
3	Mianus	28.00
4	Perryridge	16.00
5	Redwood	28.00
6	Round Hill	14.00

Question-4: Find account numbers with the highest balances for each city in the database.

Query:

```
SELECT A.account_number, A.balance, B.branch_city from account A
JOIN branch B on (A.branch_name = B.branch_name)
WHERE balance in (select max(balance) FROM account A join branch B
on (A.branch_name = B.branch_name) group by B.branch_city);
```

129
130
131
132
133

```
SELECT A.account_number, A.balance, B.branch_city from account A
JOIN branch B on (A.branch_name = B.branch_name)
WHERE balance in (select max(balance) FROM account A join branch B
on (A.branch_name = B.branch_name) group by B.branch_city);
```

Result(RO) x

Search results

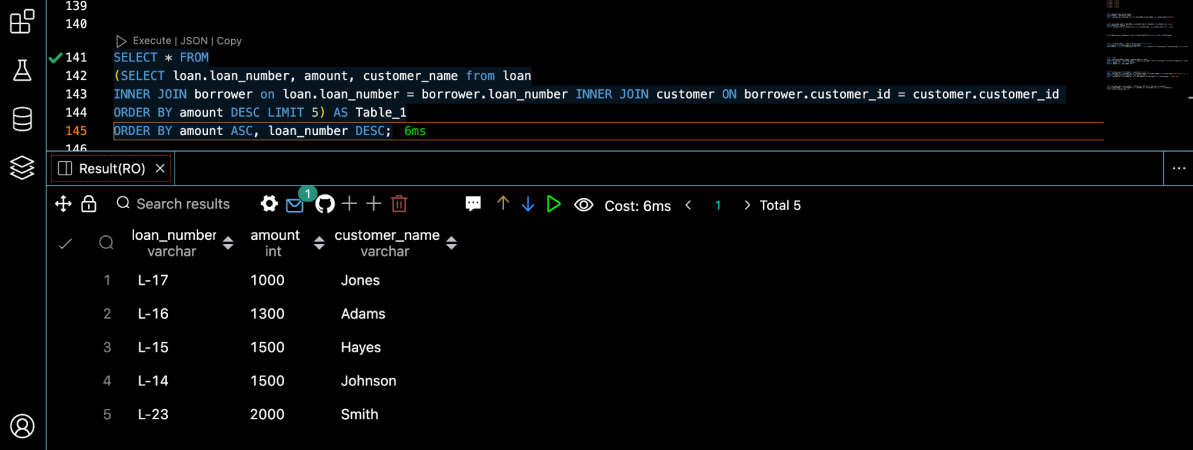
Cost: 15ms < 1 > Total 3

	account_number varchar	balance int	branch_city varchar
1	A-201	900	Brooklyn
2	A-215	700	Horseneck
3	A-222	700	Palo Alto

Question-5: Show the loan number, loan amount, and name of customers with the top 5 highest loan amounts. The data should be sorted by increasing amounts, then decreasing loan numbers in case of the same loan amount. [Hint for top 5: Check the "limit" keyword in mysql]

Query:

```
SELECT * FROM
(SELECT loan.loan_number, amount, customer_name from loan
INNER JOIN borrower on loan.loan_number = borrower.loan_number
INNER JOIN customer ON borrower.customer_id =
customer.customer_id
ORDER BY amount DESC LIMIT 5) AS Table_1
ORDER BY amount ASC, loan_number DESC;
```



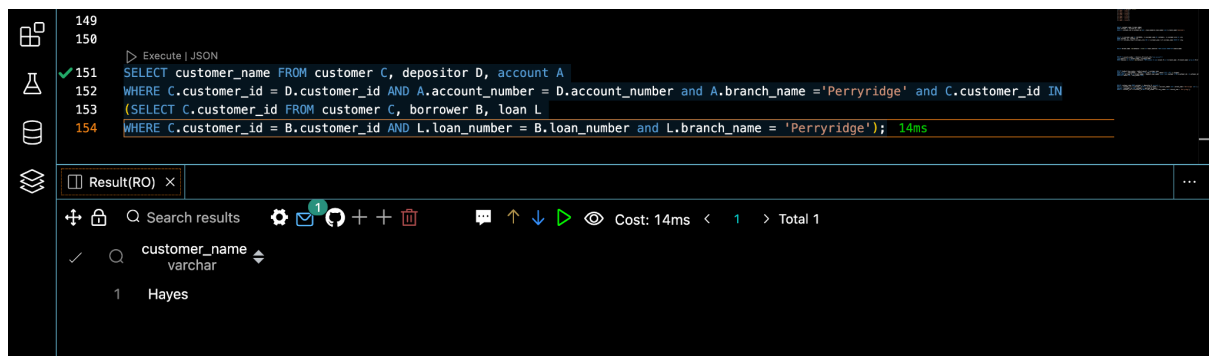
The screenshot shows a SQL IDE interface. The top pane displays a SQL query with line numbers 139 to 146. The query is:
139
140
141 SELECT * FROM
142 (SELECT loan.loan_number, amount, customer_name from loan
143 INNER JOIN borrower on loan.loan_number = borrower.loan_number INNER JOIN customer ON borrower.customer_id = customer.customer_id
144 ORDER BY amount DESC LIMIT 5) AS Table_1
145 ORDER BY amount ASC, loan_number DESC; 6ms
146
The bottom pane shows the results of the query in a table format. The table has three columns: loan_number (varchar), amount (int), and customer_name (varchar). There are 5 rows of data. The table is titled 'Result(RO)'.

	loan_number varchar	amount int	customer_name varchar
1	L-17	1000	Jones
2	L-16	1300	Adams
3	L-15	1500	Hayes
4	L-14	1500	Johnson
5	L-23	2000	Smith

Question-6: Find the names of customers with an account and also a loan at the Perryridge branch.

Query:

```
SELECT customer_name FROM customer C, depositor D, account A
WHERE C.customer_id = D.customer_id AND A.account_number =
D.account_number and A.branch_name ='Perryridge' and
C.customer_id IN
(SELECT C.customer_id FROM customer C, borrower B, loan L
WHERE C.customer_id = B.customer_id AND L.loan_number =
B.loan_number and L.branch_name = 'Perryridge');
```



The screenshot shows a SQL IDE interface. The top pane displays a SQL query with line numbers 149 through 154. The query is a complex join involving customer, depositor, account, borrower, and loan tables, filtered for the 'Perryridge' branch. The bottom pane shows the execution results in a table format. The table has one column, 'customer_name', and one row with the value 'Hayes'. The interface also shows a search bar, a toolbar with various icons, and a status bar indicating the cost of the query (14ms) and the total number of rows (1).

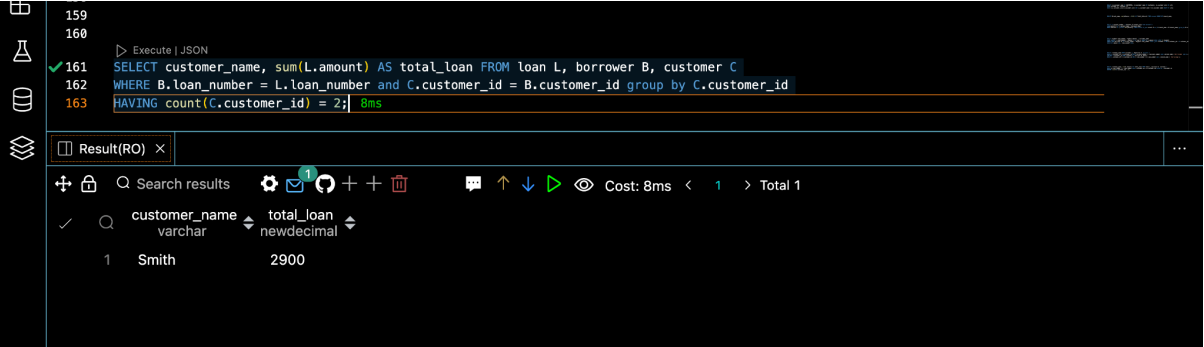
```
149
150
151 SELECT customer_name FROM customer C, depositor D, account A
152 WHERE C.customer_id = D.customer_id AND A.account_number = D.account_number and A.branch_name ='Perryridge' and C.customer_id IN
153 (SELECT C.customer_id FROM customer C, borrower B, loan L
154 WHERE C.customer_id = B.customer_id AND L.loan_number = B.loan_number and L.branch_name = 'Perryridge'); 14ms
```

customer_name
Hayes

QUESTION-7: Find the total loan amount of all customers having at least 2 loans from the bank. Show in format customer name, total_loan.

QUERY:

```
SELECT customer_name, sum(L.amount) AS total_loan FROM loan L,
borrower B, customer C
WHERE B.loan_number = L.loan_number and C.customer_id =
B.customer_id group by C.customer_id
HAVING count(C.customer_id) = 2;
```



The screenshot shows a SQL query execution interface. The query is as follows:

```
159
160
161 SELECT customer_name, sum(L.amount) AS total_loan FROM loan L, borrower B, customer C
162 WHERE B.loan_number = L.loan_number and C.customer_id = B.customer_id group by C.customer_id
163 HAVING count(C.customer_id) = 2; 8ms
```

The results are displayed in a table with the following columns: customer_name (varchar) and total_loan (newdecimal). The table contains one row with the value 'Smith' for customer_name and '2900' for total_loan.

customer_name	total_loan
Smith	2900